

Федеральное государственное автономное образовательное учреждение высшего
образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии
и компьютерной техники

Отчет по проекту по дисциплине
«Компьютерные сети»

Вариант № 2

Выполнил:

Мартышов Данила Викторович,

Группа Р3307

Проверил:

Зинатулин Артём Витальевич

Санкт-Петербург, 2026

Содержание

Содержание.....	2
Задание.....	3
Ход работы.....	4
Решение для отказоустойчивости:.....	4
Независимость от ML модели:.....	4
Протокол:.....	4
Общее про реализацию:.....	5
Вывод:.....	7

Задание

Пользовательский интерфейс:

- Сервер

Сервер работает постоянно, принимает запросы от клиентов, сервер имеет может обучать несколько алгоритмов параллельно. Клиенты на уровне протокола выбирают, какую задачу они считают.

- Клиент

Клиент запускается по запросу, выполняет вычисления и процесс на клиенте умирает.

Требования к реализации:

- Необходимо реализовать свой протокол поверх TCP/UDP, обосновать выбор, написать спецификацию протокола.

Усложнения:

- Фреймворк, не зависящий от архитектуры ML-модели (PyTorch только), нужно включить также передачу архитектуры модели по сети
- На уровне инфраструктуры предложить решение, чтобы сервер не был единственной точкой отказа системы

Ход работы

Решение для отказоустойчивости:

В изначальной архитектуре сервер, на котором лежит модель с самыми последними весами, является единой точкой отказа системы, и если он упадет, будет грустно. В виде решения к этой проблеме предлагается система из нескольких узлов сервера, в данном случае из двух, в конфигурации master - standby. Master принимает подключения от клиентов, и периодически отсылать свежие данные на standby. Standby же в свою очередь будет применять изменения, полученные от мастера, и хранить временную метку последнего контакта с мастером. Если standby не будет получать сообщений об активности master в течение достаточно долгого времени (15 секунд в данной реализации), выполняется promotion standby узла до master, и он будет принимать подключения от клиентов. При этом если в процессе репликации произошел сбой, и на standby не дошел последний снимок состояния, при переключении standby как master прогон текущего раунда начнется снова, то есть каких-то критичных потерь система не потерпит, клиентам придется просто переиграть раунд обучения.

Независимость от ML модели:

Для того чтобы обеспечить независимость клиентов от архитектуры модели, обучаемой на сервере, используется TorchScript, компилятор PyTorch. Он преобразует модель в платформонезависимое представление, которое затем можно сохранить в файл или загрузить в программу, не имея в коде исходного класса модели. Это самое платформонезависимое представление содержит в себе граф вычислений, имена и формы параметров, а также их начальные значения. Но для того чтобы у клиентов была актуальная модель, им также надо пересылать актуальные веса. Таким образом, клиенты будут получать независимое представление, которое они смогут десериализовать и загрузить, без всяких импортов классов моделей, а также актуальные веса, которые используются для перезаписи дефолтных значений из представления. В итоге у клиентов будет модель, которую они смогут обучить на своих данных, и затем отправить обновленные веса на сервер.

Протокол:

В этом разделе находится общее описание протокола, используемого в системе. Полноценная детализированная спека находится в репозитории в файле `protocol_spec.md`.

Протокол FL/1, используемый в системе - это прикладной протокол, используемый для организации федеративного обучения модели машинного обучения. Этот протокол работает поверх TCP, ибо UDP ненадежный, а веса модели - весьма чувствительные данные, и если в процессе обмена пропадет запрос, или запросы придут не в том порядке, в котором были задуманы, их нельзя будет нормально десериализовать, а если можно, то получим бредик, и клиентам придется переобучать модель до момента успешной передачи весов.

Фрейм состоит из заголовка и тела. Заголовки содержат в себе идентификатор протокола (волшебное число) (4 байт), тип сообщения (1 байт), и длину тела в байтах (4 байт).

Типы сообщений:

Код	Имя	Направление	Описание
0x01	JOIN	Client → Server	Присоединиться к задаче или получить список задач
0x02	SUBMIT	Client → Server	Отправить обновлённые веса
0x03	STATUS	Client → Server	Опросить статус раунда
0x10	TASK_INFO	Server → Client	Информация о задаче (arch + weights)
0x11	WEIGHTS	Server → Client	Новые глобальные веса после агрегации
0x12	PENDING	Server → Client	Раунд ещё не завершён
0x13	DONE	Server → Client	Обучение завершено
0x20	HEARTBEAT	Primary → Backup	Подтверждение активности primary
0x21	REPL_STATE	Primary → Backup	Полный снимок состояния

0xF0	ACK	Server → Client	Подтверждение приёма (ожидаем quorum)
0xFF	ERROR	Server → Client	Ошибка

Общее про реализацию:

Как и указано в тз, клиенты - короткоживущие процессы, которые подключаются к серверу, обучают модель на своих данных, отправляют веса на сервер, и завершаются. Сервер работает постоянно, принимает обновления от клиентов, агрегирует их, и хранит глобальную модель.

Модель, используемая в данной реализации - двухслойная полносвязная сеть для классификации изображений MNIST. Агрегация весов ведется с помощью алгоритма Federated Averaging.

Обучение модели базируется на задачах, которые создаются на сервере. Клиенты могут получить список доступных к выполнению задач с помощью типа сообщений JOIN, и выбрать, какую из них выполнить, указав id задачи в теле сообщения. После этого клиенты получают данные модели от сервера, выполняют обучение, и возвращают результаты на сервер. В задаче может быть несколько раундов обучения, а также можно задать минимальное количество клиентов, которые должны обработать задачу, чтобы агрегировать полученные данные. Клиенты могут одновременно обрабатывать одну задачу. В случае если два клиента обрабатывают одну задачу, они обрабатывают разные датасеты. При этом если один из клиентов закончил обрабатывать быстрее, он ожидает, пока не закончит второй клиент, прежде чем получить новые агрегированные значения от сервера и перейти к новому раунду обучения.

Для более наглядной визуализации в системе присутствует дашборд, в котором можно посмотреть доступные задачи, прогресс по ним, а также метрики по обучаемой модели.

Вывод:

В ходе работы над проектом разработал собственный протокол, работающий поверх TCP, включая структуру фреймов протокола, а также различные типы сообщений, поддерживаемых протоколом. Также создал систему федеративного обучения, использующую протокол как для клиент-серверного взаимодействия, так и для репликации состояния системы с целью повышения отказоустойчивости.